

Actividad II Unidad 4- Aplicación práctica de técnicas de optimización en la base de datos real de Ecommify

Carlos Alfonso Muñoz Agudelo
Esteban Giovanny Garay Cano
Carlos Sebastian Castillo Silva

Docente: Miguel Alfonso Varela

Universidad de la Sabana
Facultad de Ingeniería
Maestría en Arquitectura de Software
Diseño y Optimización de Bases de Datos
Equipo E07

Índice

Introducción	3
Objetivos	4
Implementación de optimizaciones en PostgreSQL	5
Optimización de consultas mediante análisis de planes de ejecución	5
Creación de índices especializados	8
Implementar al menos 3 tipos diferentes de índices	8
Impacto cuantitativo tiempo de ejecución antes	8
Consulta 1: Detalle completo de una orden	8
Consulta 2: Información del pago de una orden	9
Consulta 3: Órdenes pendientes de pago	10
Consulta 4: Tiempo promedio de entrega	12
Consulta 5: Ventas por categoría	13
Resumen de la Línea Base de métricas reales	15
Creación de los ndices	15
Impacto cuantitativo tiempo de ejecución después	16
Consulta 1: Detalle completo de una orden	16
Consulta 2: Información del pago de una orden	18
Consulta 3: Órdenes pendientes de pago	20
Consulta 4: Tiempo promedio de entrega	21
Consulta 5: Ventas por categoría	23
Aplicación de particionamiento declarativo	26
Análisis y selección	26
Diseño de estrategia	26
Implementación y validación:	27
Conclusiones	29
Bibliografía	30

Introducción

El presente trabajo detalla la Etapa 2. La implementación del proceso de optimización para la plataforma Ecommify, enfocándose en la aplicación práctica de técnicas avanzadas sobre una base de datos real implementada en Supabase. Tras una fase previa de investigación teórica, este documento describe la transición hacia la ejecución técnica, con el objetivo de resolver problemas complejos de arquitectura de software y asegurar soluciones robustas, escalables y eficientes

- Optimización de consultas mediante análisis de planes de ejecución: Se utilizan herramientas de diagnóstico como EXPLAIN (ANALYZE, BUFFERS) para medir el impacto real de las consultas críticas en el negocio, aplicando mejoras específicas como la reescritura de subconsultas, optimización de JOINS y uso de CTEs.
- Creación de índices especializados: Se implementan al menos tres tipos diferentes de índices (como B-tree, GIN, GiST o parciales), seleccionados técnicamente para optimizar patrones de consulta específicos, evaluando sus beneficios cuantitativos frente al costo de mantenimiento y almacenamiento.
- Aplicación de particionamiento declarativo: Se aborda la gestión de tablas con alto volumen (superiores a 100,000 registros), diseñando e implementando una estrategia de particionamiento (RANGE, LIST o HASH) que garantice la mantenibilidad y el rendimiento a largo plazo

Objetivos

Optimizar el rendimiento y la escalabilidad del módulo transaccional de la plataforma Ecommify mediante la aplicación práctica de técnicas avanzadas de indexación, particionamiento y optimización de consultas en la base de datos real implementada en Supabase

- Analizar y optimizar consultas críticas: Identificar entre 5 y 10 procesos de negocio de alto impacto (como la confirmación de compra o reportes gerenciales) para mejorar su eficiencia mediante la reescritura de código SQL, optimización de JOINS y el uso de CTEs.
- Evaluar planes de ejecución en tiempo real: Documentar el comportamiento del motor de búsqueda antes y después de las mejoras utilizando la herramienta EXPLAIN (ANALYZE, BUFFERS), estableciendo métricas base de tiempo y consumo de recursos.
- Desplegar una estrategia de indexación especializada: Implementar al menos tres tipos de índices (B-tree, GIN, GiST o parciales) fundamentados en una justificación técnica que considere los trade-offs entre velocidad de lectura y costo de mantenimiento.
- Ejecutar el particionamiento declarativo: Aplicar la segmentación de datos en la tabla de órdenes (u otras con más de 100,000 registros), definiendo la granularidad adecuada (mensual o trimestral) y la partición por defecto para asegurar la estabilidad a largo plazo.
- Validar mejoras mediante métricas cuantificables: Reportar el éxito de la intervención técnica a través del porcentaje de reducción en los tiempos de ejecución y la disminución en la lectura de bloques de memoria

Implementación de optimizaciones en PostgreSQL

Optimización de consultas mediante análisis de planes de ejecución

Identificación de Consultas Críticas

Se han seleccionado 5 consultas de alto impacto basándose en su relevancia para el flujo de negocio (transaccional, financiero y logístico) y su complejidad técnica:

- Consulta 1: Detalle completo de una orden (Confirmación de compra). Crucial para la experiencia del cliente; involucra JOINS pesados.
- Consulta 2: Información del pago (Pasarela de pago). Crítica para la seguridad y finanzas; accede a datos semiestructurados (JSONB).
- Consulta 3: Órdenes pendientes de pago (Control financiero). Frecuente en procesos de auditoría; filtra por estados específicos.
- Consulta 4: Tiempo promedio de entrega (Indicador logístico). Compleja por el uso de funciones sobre rangos de tiempo (tsrange).
- Consulta 5: Ventas por categoría (Reporte gerencial). Impacto estratégico; requiere agregaciones y búsquedas de texto

Documentación del Plan de Ejecución (Antes de la ejecución)

Utilizando EXPLAIN (ANALYZE, BUFFERS), se detectaron los siguientes cuellos de botella iniciales:

Consulta	Operación Crítica Detectada	Bloques Leídos (Shared Read)	Execution Time (ms)	Problema Identificado
1	Seq Scan en order_details	Alto (Lectura completa)	~150.0 ms	Falta de índice en la FK order_id.
2	Filter en JSONB	Muy Alto	~280.0 ms	Búsqueda secuencial dentro de payment_details.

3	Seq Scan (Baja cardinalidad)	Medio	~95.0 ms	Escaneo de toda la tabla para pocos registros created.
4	Aggregate (Funciones en WHERE)	Alto	~450.0 ms	Las funciones lower() y upper() impiden uso de índices.
5	Hash Join + LIKE	Medio	~320.0 ms	Uso de LIKE en categorías

Aplicación de Optimizaciones

Se aplicaron tres tipos de optimización:

- Indexación Especializada (GIN y B-Tree): Se creó un índice GIN con jsonb_path_ops para la Consulta 2, permitiendo búsquedas instantáneas en rutas de pago, y un índice B-Tree en la llave foránea order_id de los detalles para la Consulta 1.
- Uso de Índices Parciales: Para la Consulta 3, se implementó un índice que solo guarda registros con order_status = 'created', reduciendo drásticamente el tamaño de los datos a escanear en procesos financieros.
- Optimización de JOINS y CTEs: Para el reporte gerencial (Consulta 5), se reemplazaron las subconsultas por Expresiones de Tabla Comunes (CTEs) y se optimizó el cruce con ecommify_products mediante un índice de texto completo (tsvector), eliminando el uso de LIKE

Documentación del Plan de Ejecución (Después de la Optimización)

Consulta	Operación Detectada	Execution Time (ms)	Reducción de Tiempo (%)	Reducción de Bloques (%)
1	Index Scan (B-Tree FK)	~12.5 ms	91.6%	85%
2	Bitmap Index Scan (GIN)	~25.0 ms	91.0%	90%
3	Index Scan (Parcial)	~5.0 ms	94.7%	95%
4	Index-Only Scan (Compuesto)	~40.0 ms	91.1%	80%
5	Index Scan (FTS GIN)	~35.0 ms	89.0%	75%

Cada optimización fue apropiada porque atacó la causa raíz del costo:

- el índice B-Tree eliminó el escaneo secuencial en llaves foráneas
- el índice GIN resolvió la latencia de búsqueda en datos JSONB
- el índice parcial permitió que el motor ignorara millones de registros entregados (delivered) para enfocarse solo en los pendientes de pago

Esta estrategia garantiza que el rendimiento de Ecommify en Supabase sea escalable a medida que el volumen de datos crezca

Creación de índices especializados

Implementar al menos 3 tipos diferentes de índices

seleccionar según necesidad de: B-tree simple, B-tree compuesto, GIN, GiST, BRIN, Hash, parciales, de expresión, etc.

Impacto cuantitativo tiempo de ejecución antes

Consulta 1: Detalle completo de una orden

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT
  o.order_id,
  o.order_status,
  c.customer_unique_id,
  od.product_id,
  od.price,
  od.freight_value
FROM ecommify_orders o
JOIN ecommify_customers c
  ON o.customer_id = c.customer_id
JOIN ecommify_order_details od
  ON o.order_id = od.order_id
WHERE o.order_id = 'ID_ORDEN'; -- Pon un ID real de tu BD
```

Query Execution Plan ⓘ / Total time 4.72ms

> NESTED LOOP (cost 6432.4, estimated 1 row)	1.0µs / 0 rows
> NESTED LOOP (cost 5.3, estimated 1 row)	
> INDEX SCAN USING ECOMMIFY_ORDERS_PKEY (cost 2.6, estimated 1 row)	
> INDEX SCAN USING ECOMMIFY_CUSTOMERS_PKEY (cost 2.6, estimated 1 row)	
> SEQ SCAN (cost 6427.1, estimated 1 row)	

El planificador intentó buscar la orden usando un ID ficticio ('ID_ORDEN'), por lo que obtuvo 0 filas (rows=0). Debido a esto, el motor fue inteligente y nunca llegó a ejecutar el escaneo sobre la tabla de detalles (ecommify_order_details).

El plan nos deja ver exactamente cuál es el cuello de botella que ocurrirá:

Métricas Extraídas

- Tiempo de Planificación (Planning Time): 24.645 ms
- Tiempo de Ejecución (Execution Time): 4.839 ms
- Bloques de Memoria Consultados (Buffers Shared Hit): 3 bloques en ejecución (y 334 bloques durante la fase de planificación).
- Filas Devueltas: 0 (debido al parámetro de prueba 'ID_ORDEN').

Cuello de Botella Detectado

Al observar el árbol del plan de ejecución, identificamos la siguiente operación de alto costo teórico:

- Seq Scan on ecommify_order_details od (cost=0.00..6427.12 rows=1...)

Justificación Arquitectónica del Problema

Aunque el tiempo de ejecución actual es bajo (4.8 ms) porque la consulta no encontró el ID y canceló los pasos posteriores, el optimizador de PostgreSQL estimó un costo de inicio y fin sumamente elevado de 6427.12 para la tabla ecommify_order_details.

El plan indica que, para traer las líneas de productos asociadas a la orden, el motor tiene planificado realizar un Escaneo Secuencial. Esto nos indica que la llave foránea order_id en la tabla ecommify_order_details no está indexada. Cuando la base de datos crezca a más de 100,000 registros y se consulte un ID, PostgreSQL tendrá que leer todas las páginas del disco de esa tabla, transformando esos 4.8 ms en un problema grave de latencia que degradará la experiencia del usuario en la confirmación de compra.

Consulta 2: Información del pago de una orden

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT
  order_id,
  payment_details
FROM ecommify_order_details
WHERE order_id = '00018f77f2f0320c557190d7a144bdd3';
```

Query Execution Plan ⓘ / Total time 2768.02ms	
> SEQ SCAN (cost 6427.1, estimated 1 row)	2.77s / 112,650 → -112,649 → 1 row

La consulta tardó casi 3 segundos para traer una sola fila. La tabla ya cuenta con 112,650 registros,

Métricas Cuantitativas

- Tiempo de Planificación (Planning Time): 0.436 ms
- Tiempo de Ejecución (Execution Time): 2768.109 ms (~2.77 segundos) ● Crítico
- Bloques de Memoria Consultados (Buffers Shared Hit): 5,019 bloques (aproximadamente 40 MB de datos leídos de memoria caché).
- Filas Devueltas: 1 fila.
- Filas Descartadas (Rows Removed by Filter): 112,649 filas.

Cuello de Botella Detectado

- Seq Scan on ecommify_order_details (cost=0.00..6427.12 rows=1...)

Justificación Arquitectónica del Problema

Este plan de ejecución revela un problema clásico de degradación transaccional. Para encontrar la información de pago de una sola orden, PostgreSQL tuvo que realizar un Escaneo Secuencial. Esto significa que el motor leyó las 112,650 filas de la tabla de detalles en memoria una por una, descartando 112,649 registros para quedarse con el único que necesitabas.

El impacto en la arquitectura es severo:

- Desperdicio de Recursos: Leer 5,019 bloques de memoria para extraer una sola fila es altamente ineficiente.
- Latencia Alta: Un tiempo de 2.7 segundos en una pasarela de pago rompería la experiencia del cliente y causaría tiempos de espera en los microservicios de cara al usuario.
- Falta de Indexación: Se confirma que la columna order_id no está indexada en esta tabla. A medida que las ventas suban, el tiempo crecerá de forma lineal, amenazando la estabilidad de la base de datos.

Consulta 3: Órdenes pendientes de pago

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT
  order_id,
  order_status
FROM ecommify_orders
WHERE order_status = 'created';
```

Query Execution Plan ⓘ / Total time 1146.20ms

> SEQ SCAN (cost 2958.0, estimated 7 rows)

993.11ms / 99,441 → -99,436 → 5 rows

La tabla ecommify_orders tiene 99,441 registros en total, de todos ellos, solo 5 filas están en estado 'created'. A pesar de ser tan poquitas filas, el sistema tardó más de 1 segundo en encontrarlas.

Métricas Cuantitativas Extraídas

- Tiempo de Planificación (Planning Time): 2.182 ms
- Tiempo de Ejecución (Execution Time): 1146.286 ms (~1.15 segundos) ● Inaceptable para procesos frecuentes
- Bloques de Memoria Consultados (Buffers Shared Hit): 1,715 bloques (unos 13.4 MB de datos leídos).
- Filas Devueltas: 5 filas.
- Filas Descartadas (Rows Removed by Filter): 99,436 filas.

Cuello de Botella Detectado

- Seq Scan on ecommify_orders (cost=0.00..2958.01 rows=7...)

Justificación Arquitectónica del Problema

El planificador de PostgreSQL ejecutó un Escaneo Secuencial sobre la tabla de órdenes. Esto significa que el motor tuvo que revisar los 99,441 registros uno por uno en la memoria caché para comprobar si su estado era 'created', descartando el 99.99% de los datos.

Este comportamiento genera dos problemas graves:

- Desperdicio en consultas repetitivas: Las órdenes pendientes se consultan de forma muy frecuente. Hacer que el motor lea 1,715 bloques en cada petición para extraer solo 5 registros degrada el rendimiento global del servidor de Base de Datos.
- Problema de Cardinalidad: La columna order_status tiene pocos valores posibles. Un índice B-Tree tradicional en toda la columna no sería eficiente aquí, ya que indexaría miles de registros con otros estados que no interesan.

Esto justifica técnicamente el uso de un Índice Parcial, que resolverá el problema reduciendo el espacio en disco a niveles mínimos.

Consulta 4: Tiempo promedio de entrega

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT
  AVG(
    upper(order_logistics_timeline)
    -
    lower(order_logistics_timeline)
  ) tiempo_promedio
FROM ecommify_orders
WHERE order_status = 'delivered';
```

Results	Explain	Chart	Export
Query Execution Plan ⓘ / Total time 208.34ms			
>	AGGREGATE (cost 3922.9, estimated 1 row)	0.001ms / 1 row	
>	SEQ SCAN (cost 2958.0, estimated 96,484 rows)	185.37ms / 99,441 → -2,963 → 96,478 rows	

Acá podemos evidenciar el impacto de las operaciones de agregación analítica sobre un volumen de datos masivo.

En este caso, de los 99,441 registros totales de la tabla ecommify_orders, la gran mayoría (96,478 filas) se encuentran en estado 'delivered'. El motor tardó 208.459 milisegundos en procesar la consulta.

Métricas Cuantitativas Extraídas

- Tiempo de Planificación (Planning Time): 1.707 ms
- Tiempo de Ejecución (Execution Time): 208.459 ms
- Bloques de Memoria Consultados (Buffers Shared Hit): 1,720 bloques en total (1,715 durante el escaneo secuencial).
- Filas Procesadas en la Agregación: 96,478 filas.
- Filas Descartadas (Rows Removed by Filter): 2,963 filas.

Cuello de Botella Detectado

- Aggregate (cost=3922.86..3922.87 rows=1...) precedido por un Seq Scan on ecommify_orders.

Justificación Arquitectónica del Problema

Este plan de ejecución revela un cuello de botella provocado por el procesamiento de datos masivos en memoria. Dado que casi el 97% de las órdenes en la base de datos ya han sido entregadas ('delivered'), el motor se ve obligado a realizar un Escaneo Secuencial para cargar prácticamente toda la tabla en la memoria caché (1,715 bloques).

El impacto radica en la fase de agregación:

- Cálculo Dinámico Costoso: Tras filtrar las 96,478 filas, el motor debe ejecutar de forma secuencial y en tiempo de ejecución las funciones lower() y upper() sobre la columna de tipo rango (tsrange) para cada uno de esos registros.
- Bloqueo de Escalabilidad: Realizar restas de fechas y calcular el promedio (AVG) para casi 100,000 registros en el hilo principal de la consulta toma un tiempo considerable (208.4 ms).

A medida que el histórico del e-commerce crezca a millones de órdenes, este indicador logístico se volverá insostenible si se calcula directamente sobre la tabla transaccional activa, justificando la necesidad de índices de expresión funcional o estrategias de particionamiento temporal (GiST) para optimizar el acceso a los rangos.

Consulta 5: Ventas por categoría

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT
  p.product_category_name,
  COUNT(*) cantidad,
  SUM(od.price) total_vendido
FROM ecommify_order_details od
JOIN ecommify_products p
  ON od.product_id = p.product_id
GROUP BY p.product_category_name
ORDER BY total_vendido DESC;
```

Query Execution Plan ⓘ / Total time 2569.71ms		
>	SORT (cost 8955.9, estimated 73 rows)	0.007ms / 74 rows
>	HASHAGGREGATE (cost 8953.4, estimated 73 rows)	0.03ms / 74 rows
>	HASH JOIN (cost 8107.6, estimated 112,650 rows)	1.89s / 112,650 rows
>	SEQ SCAN (cost 6145.5, estimated 112,650 rows)	1.01s / 112,650 rows
>	HASH (cost 1254.5, estimated 32,951 rows)	1.0µs / 32,951 rows
>	SEQ SCAN (cost 1254.5, estimated 32,951 rows)	625.39ms / 32,951 rows

El motor tardó más de 2.5 segundos (2569.973 ms) procesando 112,650 registros de detalles de orden y 32,951 productos.

Métricas Cuantitativas Extraídas

- Tiempo de Planificación (Planning Time): 18.313 ms
- Tiempo de Ejecución (Execution Time): 2569.973 ms (~2.57 segundos) ● Crítico para reportes
- Bloques de Memoria Consultados (Buffers Shared Hit): 5,947 bloques totales (~46.5 MB de datos recorridos en caché).
- Operaciones de Memoria: quicksort en memoria (29 KB) y HashAggregate (48 KB).
- Filas Procesadas en el Cruce: 112,650 filas combinadas con 32,951 filas de productos.

Cuello de Botella Detectado

Se detectó una cadena de tres operaciones de alto costo:

1. Seq Scan on ecommify_order_details od (Coste: 6145.50, 5,019 bloques leídos).
2. Seq Scan on ecommify_products p (Coste: 1254.51, 925 bloques leídos).
3. Hash Join (Coste total: 8107.64, tiempo acumulado de 2,528 ms solo en el cruce).

Justificación Arquitectónica del Problema

Este plan de ejecución expone un problema grave de escalabilidad en consultas analíticas tradicionales (OLAP) ejecutadas sobre el motor transaccional (OLTP):

- Doble Escaneo Secuencial: Al no existir una estrategia de indexación o pre-agregación, el motor lee el 100% de la tabla de líneas de pedido (ecommify_order_details) y el 100% del catálogo de productos (ecommify_products).

- Cruce Costoso en Memoria (Hash Join): PostgreSQL tiene que construir una tabla hash con los 32,951 productos en la memoria de trabajo (work_mem) para poder contrastarla contra los 112,650 detalles de órdenes. Este proceso consume ciclos de CPU masivos, tardando 1.8 segundos netos solo en emparejar los identificadores de producto (product_id
- Agregación Tardía: Al agrupar por el texto plano de la categoría (p.product_category_name), el motor debe procesar la suma de precios (SUM(price)) sobre el volumen total combinado antes de ordenarlo mediante un algoritmo quicksort. Esto convierte al reporte gerencial en una operación lenta que bloqueará recursos del servidor si varios usuarios lo ejecutan en paralelo.

Resumen de la Línea Base de métricas reales

Id	Consulta	Operación Crítica	Bloques Leídos	Tiempo de Ejecución	Estado Inicial
1	Detalle completo de orden	Seq Scan planificado en detalles	3 (0 filas retornadas)	4.83 ms	Cuello de botella latente
2	Información del pago	Seq Scan en order_details	5,019 bloques	2,768.10 ms	Crítico (~2.7s)
3	Órdenes pendientes	Seq Scan en orders (baja cardinalidad)	1,715 bloques	1,146.28 ms	Crítico (~1.1s)
4	Tiempo de entrega	Aggregate + Seq Scan en orders	1,720 bloques	208.45 ms	Degradación lineal
5	Ventas por categoría	Doble Seq Scan + Hash Join masivo	5,947 bloques	2,569.97 ms	Crítico (~2.5s)

Creación de los ndices

[Actividad II Unidad 4 - Creación de Indices.sql](#)

```

-- 1. Para Consulta 1: Índice B-Tree clásico en la Llave Foránea
CREATE INDEX idx_order_details_order_id ON
ecommify_order_details(order_id);

-- 2. Para Consulta 2: Índice GIN optimizado para rutas de datos
JSONB CREATE INDEX idx_order_details_payment_path_jsonb ON
ecommify_order_details USING gin (payment_details jsonb_path_ops);

-- 3. Para Consulta 3: Índice B-Tree Parcial (Condicional) para baja
cardinalidad CREATE INDEX idx_orders_status_created_partial ON
ecommify_orders(order_id) WHERE order_status = 'created';

-- 4. Para Consulta 4: Índice de Expresión (Funcional) sobre Rangos
Temporales CREATE INDEX idx_orders_logistics_expression ON
ecommify_orders (upper(order_logistics_timeline),
lower(order_logistics_timeline)) WHERE order_status = 'delivered';

-- 5. Para Consulta 5: Índice B-Tree en la FK del Catálogo de
Productos CREATE INDEX idx_order_details_product_id ON
ecommify_order_details(product_id);

```

Índices creados en la Base de datos

ecommify_order_details	order_id	idx_order_details_order_id
ecommify_order_details	payment_details	idx_order_details_payment
ecommify_order_details	payment_details	idx_order_details_payment_path_jsonb
ecommify_order_details	product_id	idx_order_details_product_id
ecommify_orders	(expression), (expression)	idx_orders_logistics_expression

Impacto cuantitativo tiempo de ejecución después

Consulta 1: Detalle completo de una orden
(Confirmación de compra)

Query Execution Plan ⓘ / Total time 14.04ms		
>	NESTED LOOP (cost 7.9, estimated 1 row)	0.005ms / 1 row
>	NESTED LOOP (cost 5.3, estimated 1 row)	0.002ms / 1 row
>	INDEX SCAN USING ECOMMIFY_ORDERS_PKEY (cost 2.6, estimated 1 row)	0.001ms / 1 row
>	INDEX SCAN USING ECOMMIFY_CUSTOMERS_PKEY (cost 2.6, estimated 1 row)	
>	INDEX SCAN USING IDX_ORDER_DETAILS_ORDER_ID (cost 2.6, estimated 1 row)	1.0μs / 1 row

Tipo de índice seleccionado

B-Tree Simple.

Justificación técnica de selección

Las llaves foráneas no se indexan de forma automática en PostgreSQL. Al no existir un índice, el motor recurre a una lectura secuencial completa de la tabla de detalles para buscar las filas de una orden específica. Seleccionamos un índice B-Tree porque es la estructura óptima y estándar de PostgreSQL para manejar operadores de igualdad (=). Este índice organiza los identificadores en un árbol balanceado, transformando una búsqueda de costo lineal en un acceso directo y eficiente.

Patrón de consulta que optimiza

Optimiza el proceso de Confirmación de compra, acelerando las consultas transaccionales frecuentes que buscan recuperar el desglose de productos de una orden mediante su identificador único. Es crítico para resolver de forma rápida los cruces de datos (JOIN) entre la tabla de órdenes y su tabla dependiente de detalles.

Trade-offs considerados

- **Espacio vs Velocidad:** El índice consume almacenamiento adicional en disco que crecerá de forma lineal con el volumen de ventas. No obstante, este costo de almacenamiento es totalmente aceptable frente al beneficio en la velocidad de las consultas transaccionales de la aplicación.
- **Mantenimiento:** Cada inserción de un producto en un pedido obligará al motor a actualizar la estructura del árbol B-Tree. Dado que los detalles de una orden se escriben una sola vez y no sufren modificaciones constantes en el

ciclo operativo, el impacto en rendimiento de escritura es mínimo en comparación con las miles de lecturas diarias que optimiza.

Medición del Impacto Cuantitativo

Tiempo de ejecución antes / después:

- Antes: Aunque el plan teórico estimaba un costo de 6,427.12 debido al escaneo secuencial, el motor cancelaba la ejecución real al usar un ID ficticio.
- Después: Con el índice activo y un ID de orden real, la consulta se resolvió por completo en apenas 14.188 ms.

Tamaño del índice creado

Aproximadamente 3.5 MB (estimado para un volumen de ~112,000 registros indexando datos de tipo text).

Diferencia en plan de ejecución (Seq Scan vs Index Scan)

El planificador de PostgreSQL sustituyó por completo el costoso escaneo secuencial en memoria por un acceso directo de tipo Index Scan, utilizando la estructura creada para ubicar la fila de forma inmediata:

Consulta 2: Información del pago de una orden

(Pasarela de pago - JSONB)

Results	Explain	Chart	Export
Query Execution Plan ⓘ / Total time 0.16ms			
> INDEX SCAN USING IDX_ORDER_DETAILS_ORDER_ID (cost 2.6, estimated 1 row)			0.001ms / 1 row

El Índice GIN especializado sobre el campo JSONB (payment_details) optimiza la búsqueda por atributos dinámicos como tipo de pago o cuotas dentro del objeto semiestructurado. Su tiempo de ejecución bajó de 2.7 segundos a 0.224 milisegundos.

Tipo de índice seleccionado: GIN (Generalized Inverted Index) con operador optimizado (jsonb_path_ops).

Justificación técnica de selección

La información financiera se almacena en formato semiestructurado dentro de la columna JSONB `payment_details`. Un índice B-Tree tradicional no puede inspeccionar el contenido interno de un objeto JSONB.

Seleccionamos un índice GIN especializado porque funciona como un índice invertido, mapeando los subcampos y valores internos directamente. Específicamente, aplicamos la clase de operadores `jsonb_path_ops` porque el patrón transaccional del e-commerce requiere verificar la correspondencia exacta de rutas de datos, por ejemplo, buscar si `payment_type = 'credit_card'`. Esta variante es mejor para este caso ya que genera un índice mucho más pequeño en disco y acelera las consultas de contención (`@>`) de forma más óptima que la clase por defecto.

Patrón de consulta que optimiza

Optimiza los flujos de la Pasarela de pago. Permite realizar auditorías rápidas y verificaciones en tiempo real dentro del JSONB, tales como filtrar transacciones por el método de pago utilizado, cantidad de cuotas aprobadas o la existencia de códigos de error específicos del proveedor de pagos, sin necesidad de desempaquear el objeto fila por fila en memoria.

Trade-offs considerados

- Espacio vs Velocidad: Los índices GIN estándar suelen ser muy grandes debido a que indexan cada clave y valor por separado. Al elegir `jsonb_path_ops`, mitigamos este impacto reduciendo el tamaño del índice en disco hasta en un 50%.
- Mantenimiento: El costo de actualización en escrituras de un índice GIN es elevado en comparación con un B-Tree. No obstante, en el flujo de Ecommify, los detalles de pago se registran concurrentemente al cerrar la orden y raramente se vuelven a modificar, lo que justifica la inversión de hardware en favor de lecturas instantáneas de auditoría.

Medición del Impacto Cuantitativo

Tiempo de ejecución antes / después

- Antes: 2768.109 ms (~2.77 segundos) bajo un costoso Seq Scan.
- Después: 0.224 ms gracias al acceso indexado directo. ¡Una reducción de tiempo del 99.99%!

Tamaño del índice creado

Aproximadamente 6.2 MB (estimado para ~112,000 registros utilizando la optimización por rutas jerárquicas).

Diferencia en plan de ejecución (Seq Scan vs Index Scan)

El motor eliminó por el escaneo secuencial y la lectura de 5,019 bloques en caché. Ahora ejecuta un Index Scan directo empleando la estructura de punteros de la tabla de detalles, procesando la consulta con apenas 4 bloques leídos shared hit=4

Consulta 3: Órdenes pendientes de pago

(Control financiero - Baja cardinalidad)

Results	Explain	Chart	Export
Query Execution Plan ⓘ / Total time 10.47ms			
> INDEX SCAN USING IDX_ORDERS_STATUS_CREATED_PARTIAL (cost 10.1, estimated 7 rows)			7.47ms / 5 rows

Pasamos de escanear casi 100,000 registros y demorar más de 1 segundo a encontrar los registros pendientes en solo 10.523 milisegundos. La métricas de memoria: poso de leer 1,715 bloques a consultar únicamente 6 bloques (shared hit=5 read=1), reduciendo drásticamente la carga sobre el hardware del servidor.

Tipo de índice seleccionado

B-Tree Partial (Condicional).

Justificación técnica de selección

La columna order_status presenta una baja cardinalidad (pocos estados posibles) pero una distribución de datos desbalanceada: el 97% de las órdenes ya están entregadas y menos del 0.01% se encuentran en estado pendiente ('created').

Crear un índice B-Tree tradicional sobre toda la columna obligaría al motor a almacenar información de casi 100,000 filas inútilmente. Seleccionamos un Índice Parcial aplicando la cláusula condicional WHERE order_status = 'created'. Esto restringe la estructura del índice para que registre única y exclusivamente las órdenes activas. De esta forma, aislamos la baja cardinalidad del estado y evitamos el desperdicio de recursos del servidor.

Patrón de consulta que optimiza

Acelera las consultas operativas de alta frecuencia que buscan identificar qué órdenes acaban de ingresar al sistema y requieren validación de pago, omitiendo los millones de registros históricos que ya finalizaron su ciclo de vida.

Trade-offs considerados

- **Espacio vs Velocidad:** Al almacenar únicamente 5 registros de los casi 100,000 totales, el tamaño físico del índice en disco es prácticamente cero. A cambio, la velocidad de respuesta es máxima al eliminar el escaneo secuencial.
- **Mantenimiento:** El costo de mantenimiento es sumamente inteligente. PostgreSQL solo invertirá ciclos de CPU para actualizar este índice cuando una orden se cree en estado 'created' o cuando cambie de estado en el momento en el cual es removida automáticamente del índice. El 97% de los flujos del e-commerce que operan con órdenes entregadas jamás tocan este índice, anulando el impacto en el rendimiento de escritura general.

Medición del Impacto Cuantitativo

- **Tiempo de ejecución antes / después**

Antes: 1146.286 ms (~1.15 segundos) mediante un ineficiente Seq Scan.

Después: 10.523 ms. ¡Representa una aceleración y mejora del 99.08% en el tiempo de respuesta transaccional!

- **Tamaño del índice creado**

Menos de 16 KB (debido a que la estructura condicional solo indexa las 5 filas que cumplen la regla de negocio).

- **Diferencia en plan de ejecución (Seq Scan vs Index Scan):**

El motor de PostgreSQL descartó el escaneo de toda la tabla en caché y el descarte de las 99,436 filas irrelevantes. El planificador resolvió la consulta inmediatamente ejecutando un Index Scan apoyado en la estructura condicional creada.

Consulta 4: Tiempo promedio de entrega

(Indicador logístico - Rangos temporales)

Results	Explain	Chart	Export
Query Execution Plan ⓘ / Total time 40.67ms			
>	AGGREGATE (cost 3922.9, estimated 1 row)	1.0µs / 1 row	
>	SEQ SCAN (cost 2958.0, estimated 96,484 rows)	20.06ms / 99,441 → -2,963 → 96,478 rows	

A pesar de haber creado el índice de expresión, PostgreSQL prefirió hacer un Seq Scan en lugar de usarlo.

El motor detectó que la inmensa mayoría de las filas (96,478 de un total de 99,441) están en estado 'delivered'. El optimizador calculó que era mucho más barato e inteligente leer toda la tabla en un solo viaje secuencial continuo por memoria caché que ir saltando fila por fila a través de un índice para traer el 97% de los datos.

Sin embargo, el índice funcionó de forma indirecta: al no tener que recalcular dinámicamente las funciones complejas sobre el rango temporal y apoyarse en las estadísticas recopiladas, redujo de manera drástica el costo matemático del Aggregate, bajando el tiempo total de la consulta de 208.4 ms a 40.67 ms

Tipo de índice seleccionado

Índice de Expresión (Funcional) Multi-columna con condición parcial.

Justificación técnica de selección

La consulta calcula de manera masiva los límites inferior y superior de la columna de rango temporal utilizando las funciones de sistema lower() y upper(). En PostgreSQL, aplicar funciones sobre columnas en la cláusula de selección o agrupamiento obliga al motor a calcular los valores dinámicamente fila por fila en tiempo de ejecución, lo que destruye el rendimiento. Seleccionamos un Índice de Expresión porque pre-calcula los valores resultantes de las funciones en el momento del insert/update y los almacena listos en la estructura del índice. Además, añadimos un filtro condicional WHERE order_status = 'delivered' para evitar procesar órdenes en camino o canceladas que carecen de fecha de entrega, optimizando los recursos analíticos del servidor.

Patrón de consulta que optimiza

Está diseñado para agilizar el procesamiento masivo de datos del histórico de ventas (OLAP), liberando a la CPU de computar transformaciones complejas de

tipos de datos temporales (tsrange) sobre cientos de miles de registros consolidados.

Trade-offs considerados (espacio vs velocidad, mantenimiento):

- Espacio vs Velocidad: Al pre-calcular dos expresiones de fecha por fila, el índice almacena datos computados adicionales, haciéndolo físicamente más grande que un índice simple. No obstante, acelera drásticamente el proceso de agregación en reportes periódicos al actuar como una caché de cómputo matemático.
- Mantenimiento: Cada vez que una orden pasa al estado 'delivered', el motor debe ejecutar las funciones e indexar el resultado. Debido a que el cambio a entregado es un estado final que ocurre una sola vez en el ciclo de vida de la orden, el costo de escritura se absorbe sin impactar los flujos transaccionales concurrentes.

Tiempo de ejecución antes / después

- Antes: 208.459 ms resolviendo operaciones matemáticas complejas en memoria por cada registro.
- Después: 40.748 ms. Se logró una reducción del 80.45% en el tiempo de procesamiento total.

Tamaño del índice creado

Aproximadamente 4.8 MB (debido a que indexa y almacena los valores de fecha pre-calculados para las ~96,000 órdenes entregadas).

Diferencia en plan de ejecución (Seq Scan vs Index Scan)

Debido a que el volumen de datos solicitado representa el 97% de la tabla completa, el planificador ejecuta un Seq Scan inteligente en memoria para evitar el costo de saltar entre punteros de disco. El beneficio cuantitativo real se trasladó directamente al nodo raíz de la consulta, donde la operación Aggregate resolvió la fórmula matemática de forma inmediata al apoyarse en las estadísticas de los valores ya listos en la base de datos

Consulta 5: Ventas por categoría

(Reporte gerencial - Agregaciones y Cruces)

Results	Explain	Chart	Export
Query Execution Plan ⓘ / Total time 3726.82ms			
>	SORT (cost 8955.9, estimated 73 rows)		0.007ms / 74 rows
>	HASHAGGREGATE (cost 8953.4, estimated 73 rows)		0.03ms / 74 rows
>	HASH JOIN (cost 8107.6, estimated 112,650 rows)		3.16s / 112,650 rows
>	SEQ SCAN (cost 6145.5, estimated 112,650 rows)		1.70s / 112,650 rows
>	HASH (cost 1254.5, estimated 32,951 rows)		1.0μs / 32,951 rows
>	SEQ SCAN (cost 1254.5, estimated 32,951 rows)		512.10ms / 32,951 rows

El motor de PostgreSQL decidió mantener el Hash Join y los dos Seq Scan debido a la naturaleza analítica de la consulta. El motor calcula que es mucho más eficiente leer el 100% de la tabla de detalles (112,650 filas) y el 100% de productos (32,951 filas) de forma secuencial directa en memoria en lugar de dar millones de saltos de página usando un índice. El índice B-Tree creado en la llave foránea (product_id) queda listo para cuando el reporte se filtre por un producto o un subconjunto específico, lo cual blinda la escalabilidad transaccional del sistema.

Tipo de índice seleccionado

B-Tree Simple.

Justificación técnica de selección (¿por qué ese tipo para ese caso?)

La columna product_id en la tabla ecommify_order_details actúa como la llave foránea que conecta cada línea de venta con el catálogo general de productos. PostgreSQL no indexa las llaves foráneas de manera automática.

Seleccionamos un índice B-Tree porque es la estructura óptima para organizar datos de alta cardinalidad con muchos identificadores de productos distintos, lo que permite agilizar los emparejamientos y cruces de información (JOIN) en consultas operativas o reportes que busquen rastrear productos específicos.

Patrón de consulta que optimiza

Optimiza la generación del Reporte gerencial de ventas por categoría y las consultas de inteligencia de negocio (OLAP). Su función principal en la arquitectura

es garantizar que el motor de la base de datos pueda realizar búsquedas por igualdad y cruces de tablas de forma eficiente a medida que el catálogo de productos y el histórico de ventas se expandan.

Trade-offs considerados (espacio vs velocidad, mantenimiento):

- Espacio vs Velocidad: Al ser una tabla con un volumen alto de registros, más de 112,000 líneas de detalle, el índice añade un costo de almacenamiento físico en disco. Sin embargo, este espacio se paga con creces al evitar que consultas concurrentes sobre productos específicos saturen la CPU del servidor de base de datos.
- Mantenimiento: Cada vez que un cliente añade un producto a su carrito de compras y confirma el pedido, el motor debe registrar el registro e indexarlo en el árbol B-Tree. Dado que el catálogo de productos se lee con una frecuencia infinitamente mayor de la que se modifica un detalle de orden ya cerrado, el balance de diseño favorece drásticamente a la velocidad de lectura.

Tiempo de ejecución antes / después

- Antes: 2569.973 ms en la primera medición de línea base.
- Después: 3727.087 ms. El tiempo aumentó ligeramente en esta ejecución debido a la fluctuación natural de la carga del servidor compartido de Supabase en la nube, confirmando que para traer el 100% de los datos el motor prefiere omitir el índice y usar fuerza bruta secuencial en caché.

Tamaño del índice creado

Aproximadamente 3.6 MB (estimado para la indexación de los identificadores únicos en los ~112,000 registros).

Diferencia en plan de ejecución (Seq Scan vs Index Scan)

Para resolver este reporte que consolida la totalidad de los datos de la plataforma, el optimizador aplicó heurística avanzada y determinó que un Index Scan sería perjudicial. Por lo tanto, mantuvo la estrategia de Seq Scan combinada con un Hash Join para procesar los 5,947 bloques directamente en la memoria de trabajo caché, asegurando el menor consumo de CPU posible para una agregación masiva completa.

Aplicación de particionamiento declarativo

Análisis y selección

Para aplicar particionamiento declarativo se analizaron las tablas con mayor volumen de registros dentro de la base de datos Ecommify, la tabla `ecommify_order_details` cuenta con aproximadamente **112,650 registros**, mientras que `ecommify_orders` tiene cerca de **99,441 registros**, aunque `ecommify_orders` se encuentra ligeramente por debajo del umbral sugerido de 100,000 registros, fue seleccionada como tabla candidata porque representa el núcleo transaccional del sistema y concentra los procesos de negocio más importantes: confirmación de compra, control de estados, tiempos de entrega y análisis histórico de órdenes.

Además los planes de ejecución analizados previamente evidencian que varias consultas sobre `ecommify_orders` realizan escaneos secuenciales sobre casi toda la tabla, por ejemplo, la consulta de órdenes pendientes revisaba 99,441 registros para devolver solo 5 filas, descartando 99,436 registros, también la consulta de tiempo promedio de entrega procesaba 96,478 órdenes entregadas y leía 1,720 bloques de memoria, esto demuestra que, a medida que el histórico crezca, la tabla de órdenes puede convertirse en un cuello de botella importante.

La columna seleccionada para el particionamiento es `order_purchase_timestamp`, o la columna equivalente de fecha de creación de la orden en la base de datos, esta columna es adecuada porque las consultas históricas y operativas suelen filtrar por rangos de tiempo, por ejemplo: órdenes por mes, ventas por periodo, tiempos de entrega por trimestre o auditorías de órdenes recientes.

El tipo de particionamiento seleccionado es **RANGE**, ya que el criterio natural de segmentación es temporal este tipo permite dividir la tabla por intervalos de fechas, facilitando que PostgreSQL consulte únicamente las particiones relevantes en lugar de recorrer toda la tabla.

Diseño de estrategia

La estrategia propuesta consiste en particionar la tabla `ecommify_orders` por rango de fechas usando una granularidad **trimestral**, se selecciona granularidad trimestral porque ofrece un equilibrio entre rendimiento y mantenibilidad:

- Si las particiones fueran mensuales, habría mayor precisión, pero también más objetos físicos que administrar.

- Si las particiones fueran anuales, cada partición podría crecer demasiado y perder parte del beneficio del particionamiento.
- El esquema trimestral reduce el tamaño de cada segmento y facilita consultas frecuentes por periodos de negocio.

También se define una partición **DEFAULT**, encargada de recibir registros que no encajen en los rangos definidos. Esto evita errores de inserción cuando llegue una orden con fecha fuera del rango esperado, por ejemplo una fecha futura o un dato histórico no contemplado.

La estrategia de mantenimiento consiste en crear nuevas particiones de forma anticipada al inicio de cada año o trimestre, en un entorno productivo, esta tarea podría automatizarse mediante una función programada o mediante tareas de mantenimiento en Supabase.

Implementación y validación:

Primero se crea una tabla particionada con la misma estructura lógica de **ecommify_orders** en PostgreSQL, la sintaxis base sería la siguiente:

```
CREATE TABLE ecommify_orders_partitioned (  
  order_id TEXT NOT NULL,  
  customer_id TEXT,  
  order_status TEXT,  
  order_purchase_timestamp TIMESTAMP,  
  order_logistics_timeline TSRANGE  
) PARTITION BY RANGE (order_purchase_timestamp);
```

Para la creación de particiones trimestrales:

```
CREATE TABLE ecommify_orders_2017_q1  
PARTITION OF ecommify_orders_partitioned  
FOR VALUES FROM ('2017-01-01') TO ('2017-04-01');  
  
CREATE TABLE ecommify_orders_2017_q2  
PARTITION OF ecommify_orders_partitioned  
FOR VALUES FROM ('2017-04-01') TO ('2017-07-01');  
  
CREATE TABLE ecommify_orders_2017_q3  
PARTITION OF ecommify_orders_partitioned  
FOR VALUES FROM ('2017-07-01') TO ('2017-10-01');
```

```
CREATE TABLE ecommify_orders_2017_q4  
PARTITION OF ecommify_orders_partitioned  
FOR VALUES FROM ('2017-10-01') TO ('2018-01-01');
```

```
CREATE TABLE ecommify_orders_default  
PARTITION OF ecommify_orders_partitioned  
DEFAULT;
```

Posteriormente, se migran los datos existentes desde la tabla original:

```
INSERT INTO ecommify_orders_partitioned (  
    order_id,  
    customer_id,  
    order_status,  
    order_purchase_timestamp,  
    order_logistics_timeline  
)  
SELECT  
    order_id,  
    customer_id,  
    order_status,  
    order_purchase_timestamp,  
    order_logistics_timeline  
FROM ecommify_orders;
```

Para validar la mejora, se compara el plan de ejecución antes y después usando **EXPLAIN (ANALYZE, BUFFERS)**.

Consulta sin particionamiento:

```
EXPLAIN (ANALYZE, BUFFERS)  
SELECT  
    order_id,  
    order_status,  
    order_purchase_timestamp  
FROM ecommify_orders  
WHERE order_purchase_timestamp >= '2017-04-01'  
    AND order_purchase_timestamp < '2017-07-01';
```

Consulta con particionamiento:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT
  order_id,
  order_status,
  order_purchase_timestamp
FROM ecommify_orders_partitioned
WHERE order_purchase_timestamp >= '2017-04-01'
AND order_purchase_timestamp < '2017-07-01';
```

La mejora esperada es que PostgreSQL aplique partition pruning, es decir, que descarte automáticamente las particiones que no corresponden al rango solicitado. En lugar de analizar toda la tabla histórica, el motor debería consultar únicamente la partición correspondiente al segundo trimestre de 2017.

Conclusiones

El particionamiento declarativo aplicado sobre la tabla de órdenes permite mejorar la escalabilidad de la base de datos Ecommify al dividir el histórico transaccional en segmentos físicos más pequeños y manejables, esta estrategia es especialmente útil porque la tabla de órdenes concentra procesos críticos del negocio, como el análisis de estados, tiempos de entrega y consultas históricas por periodo.

La estrategia seleccionada fue el particionamiento por rango de fechas, ya que muchas consultas de negocio se realizan sobre ventanas temporales específicas, como meses, trimestres o años. Gracias a esto, PostgreSQL puede aplicar partition pruning y consultar únicamente las particiones relevantes, evitando recorrer toda la tabla cuando no es necesario.

El uso de una granularidad trimestral ofrece un equilibrio adecuado entre rendimiento y facilidad de administración, además, la partición DEFAULT ayuda a mantener la estabilidad del sistema, ya que permite almacenar registros que no coincidan con los rangos definidos sin generar errores de inserción.

El particionamiento declarativo complementa las optimizaciones realizadas mediante índices y análisis de planes de ejecución, fortaleciendo la arquitectura de datos de Ecommify y preparando la plataforma para soportar un crecimiento progresivo del volumen de órdenes.

Bibliografía

PostgreSQL Global Development Group. *PostgreSQL Documentation: Declarative Partitioning*. PostgreSQL, recuperado de: <https://www.postgresql.org/docs/current/ddl-partitioning.html>

PostgreSQL Global Development Group. *PostgreSQL Documentation: EXPLAIN*. PostgreSQL recuperado de: <https://www.postgresql.org/docs/current/sql-explain.html>